Skill Week-11

Session-21

Task 1: To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

```
kiran@k: ~/myproject
kiran@k:~$ cd ~
kiran@k:~$ rm -rf myproject
kiran@k:~$ mkdir myproject
cd myproject
kiran@k:~/myproject$ nano calculator.h
kiran@k:~/myproject$ nano calculator.c
kiran@k:~/myproject$ nano main.c
kiran@k:~/myproject$ gcc main.c calculator.c -o program
kiran@k:~/myproject$ ./program
Addition = 25
Division = 4
kiran@k:~/myproject$
```

```
GNU nano 8.7.1
#ifndef CALCULATOR_H
#define CALCULATOR_H

int add(int a, int b);
int divide(int a, int b);

#endif
```

```
GNU nano 8.7.1
#include "calculator.h"

int add(int a, int b)
{
    return a + b;
}

int divide(int a, int b)
{
    if (b == 0)
    {
        return -1;
    }

    return a / b;
}
```

```
GNU nano 8.7.1
#include <stdio.h>
#include "calculator.h"

int main()
{
    int a = 20;
    int b = 5;

    printf("Addition = %d\n", add(a, b));

    int result = divide(a, b);

    if (result == -1)
    {
        printf("Error: Cannot divide by zero\n");
    }
    else
    {
        printf("Division = %d\n", result);
    }

    return 0;
}
```

Task 2: To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

```
kiran@k:~/myproject$ nano error.c
kiran@k:~/myproject$ gcc error.c -o error
kiran@k:~/myproject$ ./error
Floating point exception   ./error
kiran@k:~/myproject$ ./program 2> error.log
Addition = 25
Division = 4
kiran@k:~/myproject$ ./program 2> error.log
Addition = 25
Division = 4
kiran@k:~/myproject$ cat error.log
kiran@k:~/myproject$
```

```
  GNU nano 8.7.1
#include <stdio.h>

int main()
{
    int a = 10;
    int b = 0;

    printf("%d\n", a / b);

    return 0;
}
```

Session-22

Task 1: To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

```
kiran@k:~/myproject$ sudo apt update
[sudo: authenticate] Password:
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:5 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [597
kB]
Get:6 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [733 kB
]
Get:7 http://archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [172 kB
]
Get:8 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [97.7
 kB]
Get:9 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [32
2 kB]
Get:10 http://archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [1
05 kB]
Get:11 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components
[201 kB]
Get:12 http://archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Component
s [1056 B]
Get:13 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [137
 kB]
Get:14 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [4
6.7 kB]
Get:15 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages
[199 kB]
Get:16 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en
[64.8 kB]
Get:17 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Component
s [55.5 kB]
Fetched 3144 kB in 4s (809 kB/s)
56 packages can be upgraded. Run 'apt list --upgradable' to see them.
kiran@k:~/myproject$ sudo apt update
Hit:1 http://security.ubuntu.com/ubuntu resolute-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu resolute InRelease
Hit:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
56 packages can be upgraded. Run 'apt list --upgradable' to see them.
kiran@k:~/myproject$ valgrind --version
valgrind-3.26.0
```

```
kiran@k:~/myproject$ nano memory.c
kiran@k:~/myproject$ gcc -g memory.c -o memory
kiran@k:~/myproject$ valgrind --leak-check=full ./memory
==1099== Memcheck, a memory error detector
==1099== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1099== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1099== Command: ./memory
==1099==
10
==1099==
==1099== HEAP SUMMARY:
==1099==     in use at exit: 20 bytes in 1 blocks
==1099==   total heap usage: 2 allocs, 1 frees, 1,044 bytes allocated
==1099==
==1099== 20 bytes in 1 blocks are definitely lost in loss record 1 of 1
==1099==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==1099==    by 0x400117E: main (memory.c:6)
==1099==
==1099== LEAK SUMMARY:
==1099==    definitely lost: 20 bytes in 1 blocks
==1099==    indirectly lost: 0 bytes in 0 blocks
==1099==      possibly lost: 0 bytes in 0 blocks
==1099==    still reachable: 0 bytes in 0 blocks
==1099==         suppressed: 0 bytes in 0 blocks
==1099==
==1099== For lists of detected and suppressed errors, rerun with: -s
==1099== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
kiran@k:~/myproject$
```

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *ptr = malloc(5 * sizeof(int));

    ptr[0] = 10;
    ptr[1] = 20;

    printf("%d\n", ptr[0]);

    return 0;
}
```

Task 2: To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

```
kiran@k:~/myproject$ nano calculator.c
kiran@k:~/myproject$ gcc calculator.c -o calculator
kiran@k:~/myproject$ echo "10 20" | ./calculator
30
kiran@k:~/myproject$ echo "10 20" | ./calculator
30
kiran@k:~/myproject$ echo "50 25" | ./calculator
75
kiran@k:~/myproject$ echo "50 25" | ./calculator
75
kiran@k:~/myproject$ nano test_cases.txt
kiran@k:~/myproject$ nano test.sh
kiran@k:~/myproject$ ./test.sh
-bash: ./test.sh: Permission denied
kiran@k:~/myproject$ chmod +x test.sh
kiran@k:~/myproject$ ./test.sh
Running Test Cases...
Test 1: PASS
Test 2: PASS
kiran@k:~/myproject$ ./test.sh > test_report.txt
kiran@k:~/myproject$ cat test_report.txt
Running Test Cases...
Test 1: PASS
Test 2: PASS
kiran@k:~/myproject$
```

```
  GNU nano 8.7.1
#include <stdio.h>

int main()
{
    int a, b;

    scanf("%d %d", &a, &b);

    printf("%d\n", a + b);

    return 0;
}
```

```
  GNU nano 8.7.1
10 20
50 25
100 200
5 7
```

```
  GNU nano 8.7.1
#!/bin/bash

echo "Running Test Cases..."

result=$(echo "10 20" | ./calculator)

if [ "$result" = "30" ]; then
    echo "Test 1: PASS"
else
    echo "Test 1: FAIL"
fi

result=$(echo "50 25" | ./calculator)

if [ "$result" = "75" ]; then
    echo "Test 2: PASS"
else
    echo "Test 2: FAIL"
fi
```